On-Line Handwritten Signature Biometrics – Report

Marcel Hofmann and Carlos Celaya

Outline

1. Introduction
2. Data
3. Feature Extraction
4. Performance Evaluation

1. Introduction

The objective of this programming task is to DEVELOP and EVALUATE an on-line handwritten signature verification system. According to the Theory sessions, signature verification systems can be divided into:

- **Off-line**: the input is a static image of the signature.
- **On-line**: the signature is acquired using a specific digital sensor which includes the static image and time signals related to the way the signature was done: x,y coordinates and pressure as a function of time.

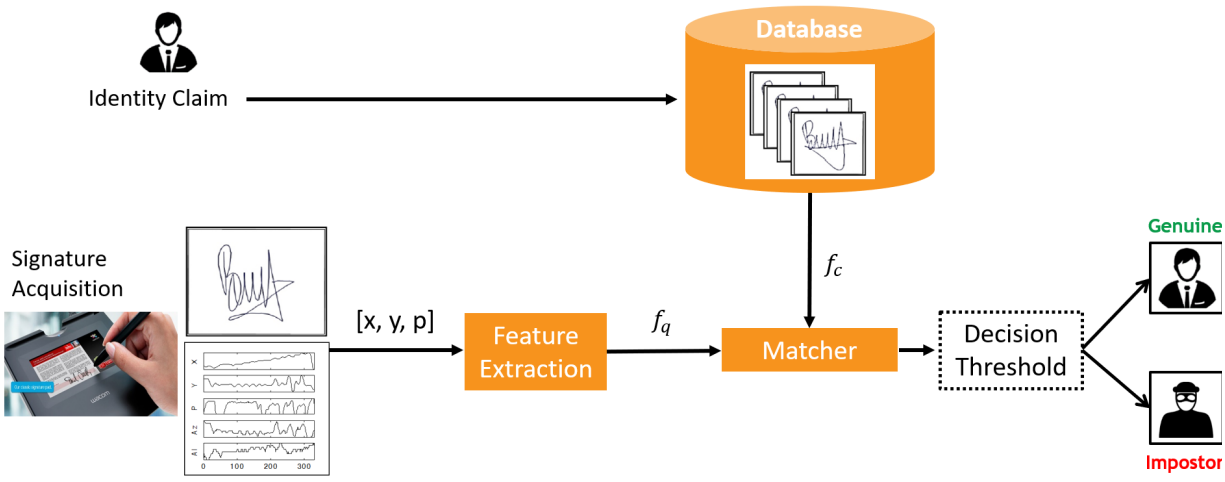
Figure 1 shows a block diagram of a typical on-line handwritten signature verification system where [x,y,p] are the time signals acquired by the sensor (Cartesian coordinates and pressure), f_q is the feature vector of the *query* signature to be compared with the f_c feature vector of the signature stored in the database (*claimed* identity).

In this programming task we will assume that the data is available (previously acquired) and already pre-processed. Thus, we will focus on the development of the following two modules:

- Feature Extraction.
- Matcher.

You must complete the tasks proposed in this report and answer the questions included.

Figure 1. Block diagram of a typical on-line handwritten signature verification system.



2. Data

For the programming task we will consider 50 users from the BiosecurID database. Each user has 28 signatures acquired in 4 sessions with a time gap of 2 months between sessions. The 28 total signatures are divided as follows: 16 are genuine (4 per session) and 12 are forgers (3 per session).

Each signature is stored in a *.mat* file, which contains three vectors (time signals) of the same length with information related to the x,y coordinates and the pressure as functions of time.

The formatting of the files is uXXXXsYYYY_sgZZZZ.mat:

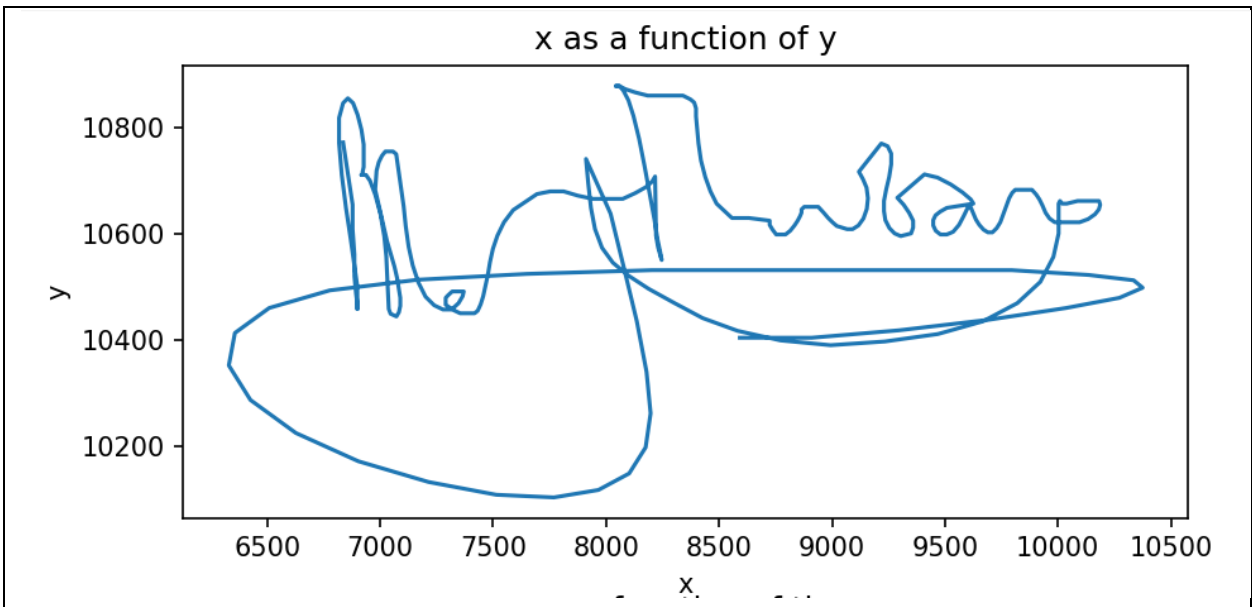
- XXXX: user number
- YYYY: session number
- ZZZZ: signature number

The GENUINE signatures of each session are those with ZZZZ=[0001,0002,0006,0007]. Signatures with ZZZZ=[0003,0004,0005] are FORGERIES.

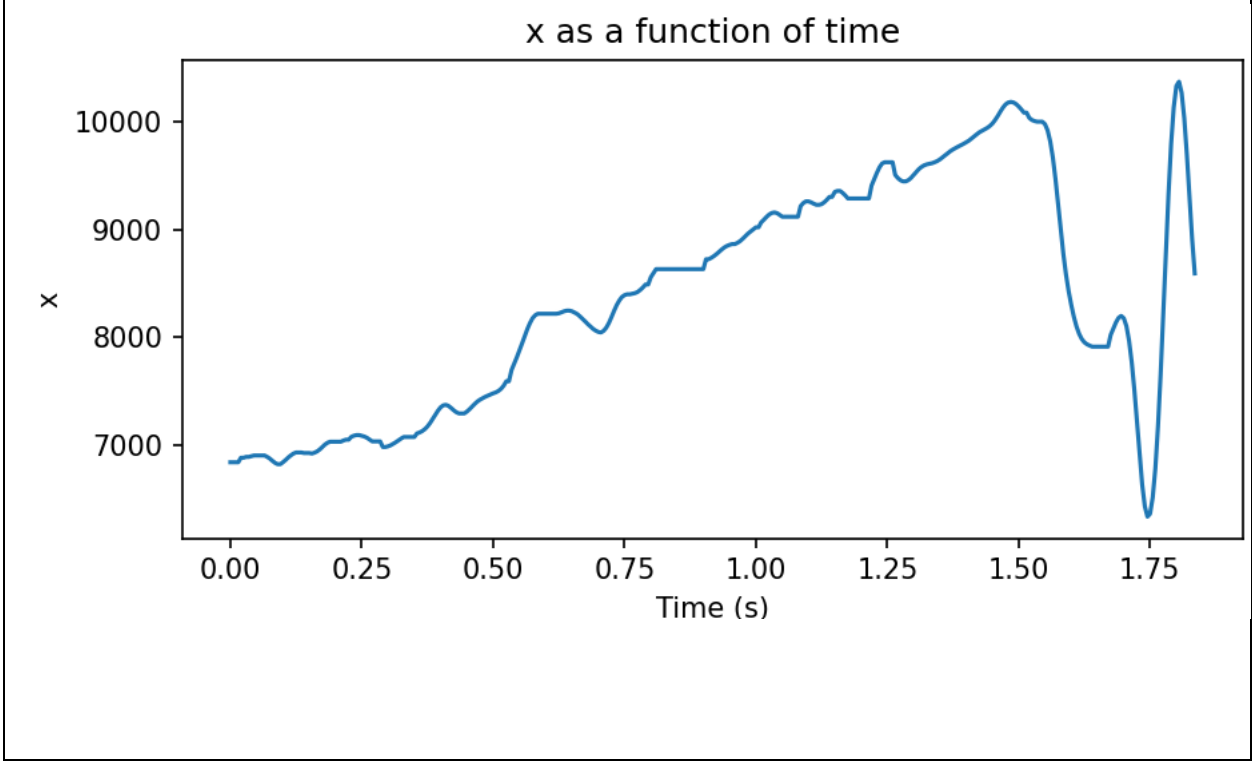
QUESTION. Choose a GENUINE signature (from a random user) and show the following representations (**you can assume that the sensor acquires samples with a sampling frequency of 200 samples/second**):

- Signal x as a function of signal y.
- Signal x as a function of time.
- Signal y as a function of time.
- Signal p as a function of time.

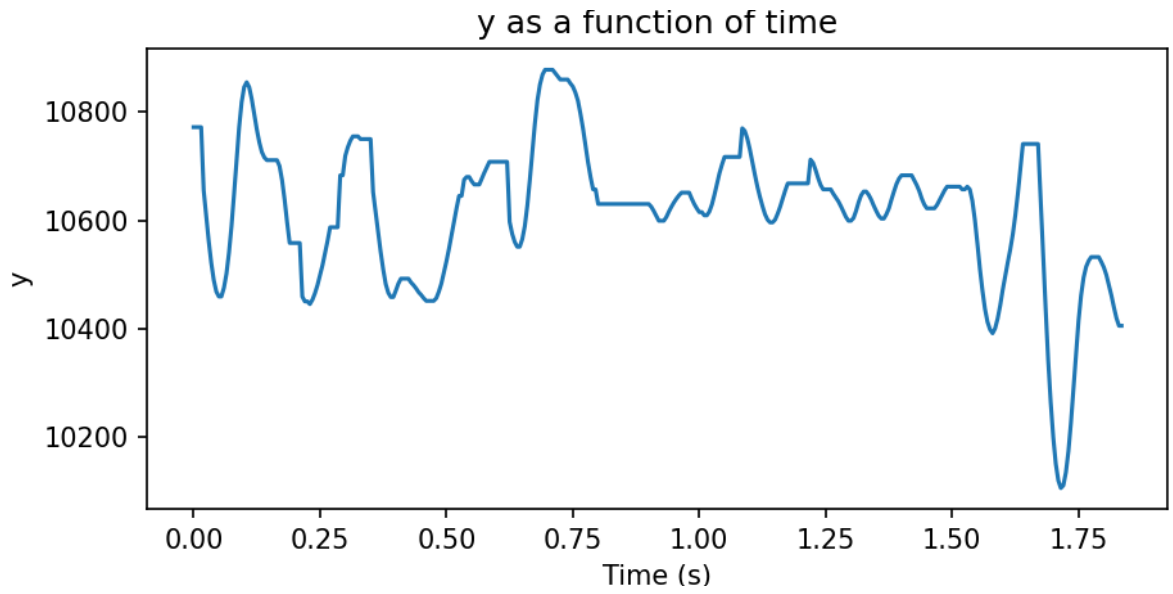
x as a function of y, *i.e.,* image of the signature



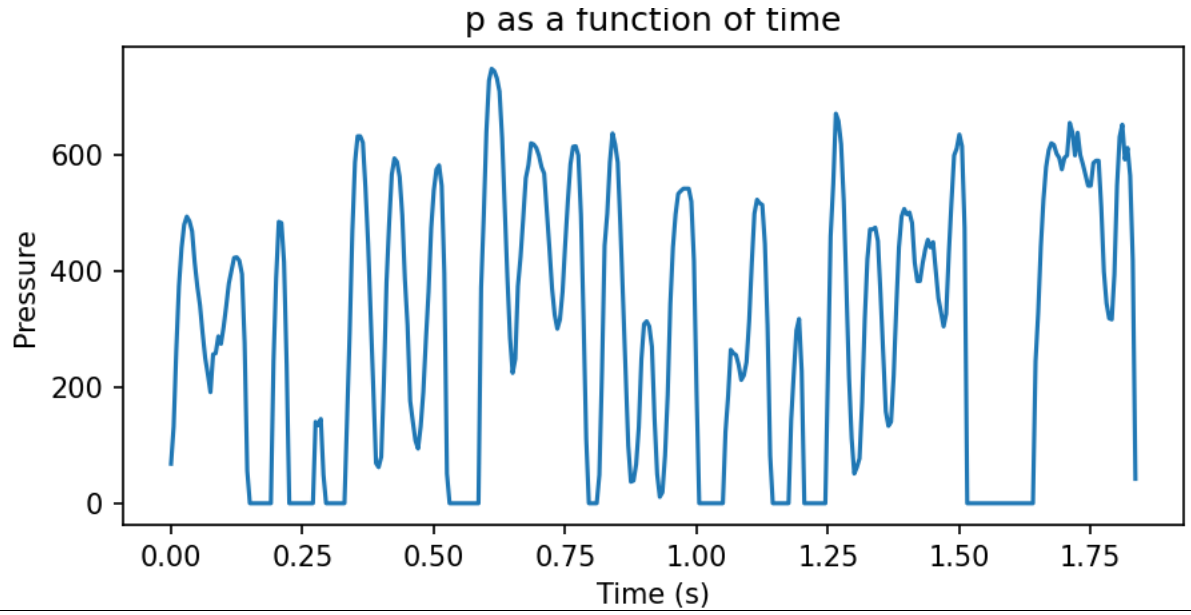
x as a function of t, *i.e.,* evolution of the time signal x with time (in seconds)



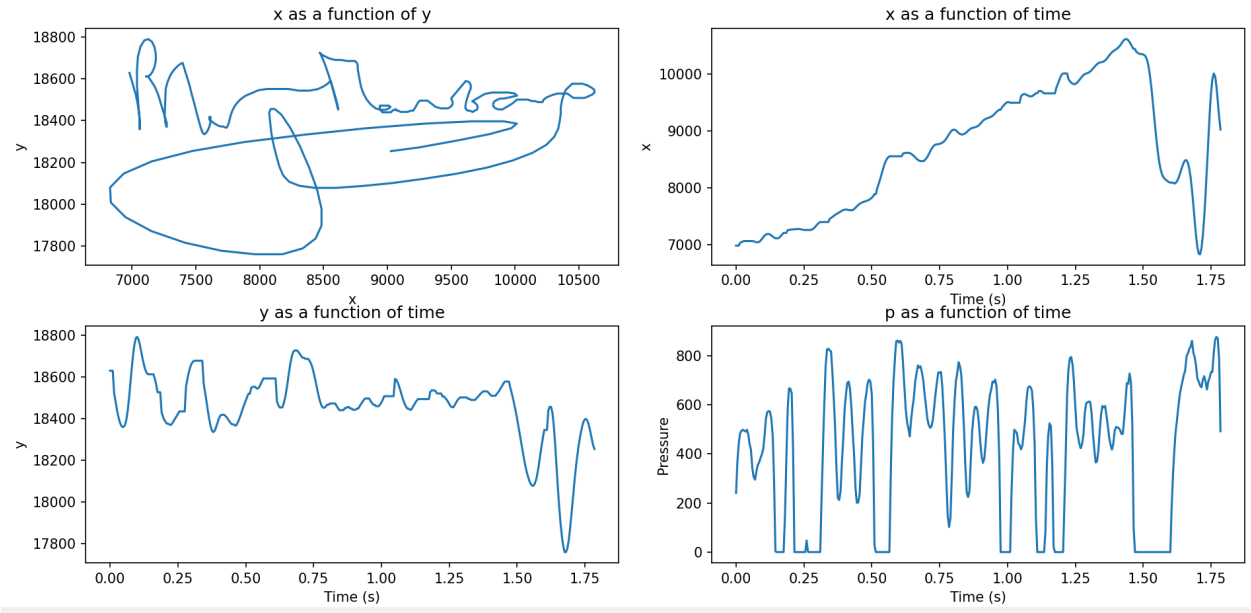
y as a function of t, *i.e.,* evolution of the time signal y with time (in seconds)



p as a function of t, *i.e.,* evolution of the time signal p with time (in seconds)



Repeat the task with another GENUINE signature of the same user and answer the following questions.



QUESTION. What is your opinion about the intra-user variability? Do they have the same time duration? Please, answer the questions properly using the explanations provided in Theory.

Well even if during the theoretical lessons we learnt that the intraclass variability can be very high, in this particular case, what can be observed is that all the graphs are very similar one to other, in terms od time durations and shapes, so from this example can be assumed that even if in some cases intravariability class can be very different this is not our case.

3. Feature Extraction

The comparison of time signals with different durations is not trivial. Therefore, to facilitate the comparison, for each signature we will extract 4 global features. So, each signature will be represented by a feature vector with a fixed size equal to 4. These global features are:

- Total duration of the signature (in seconds): T

- Number of *pen-ups* (i.e., number of times the pen was lifted). It means the number of times (not the number of samples) that p is equal to 0.
- Duration of *pen-down* T_d (i.e., signal p is different to 0) divided by the total duration T: T_d/T
- Average pressure in *pen-down* (i.e., signal p is different to 0).

You must program 4 functions to extract each of the global features:

- *T=Ttotal(x)*
- *Npu=Npenups(p)*
- *Tpd=Tpendown(p)*
- *Ppd=Ppendown(p).*

You might find useful the Python code *Part_3_feature_extraction.py* to complete this section. Then, you must program a new function, *FeatVect=featureExtractor(x,y,p)*, which receives the input data (x,y,p) of a given signature and outputs the feature vector containing the 4 global features. Based on your function *featureExtractor(),* you must generate:

1) A 3-dimensional matrix (*BiosecurIDparameters_genuine*) that includes all the feature vectors from the GENUINE signatures of the database:

- Dimension 1: number of users (1:50)
- Dimension 2: number of GENUINE signatures per user (1:16).
- Dimension 3: number of global features per signature (1:4)

2) A 3-dimensional matrix (*BiosecurIDparameters_forgery*) that includes all the feature vectors from the FORGERY signatures of the database:

- Dimension 1: number of users (1:50)
- Dimension 2: number of FORGERY signatures per user (1:12)
- Dimension 3: number of global features per signature (1:4)

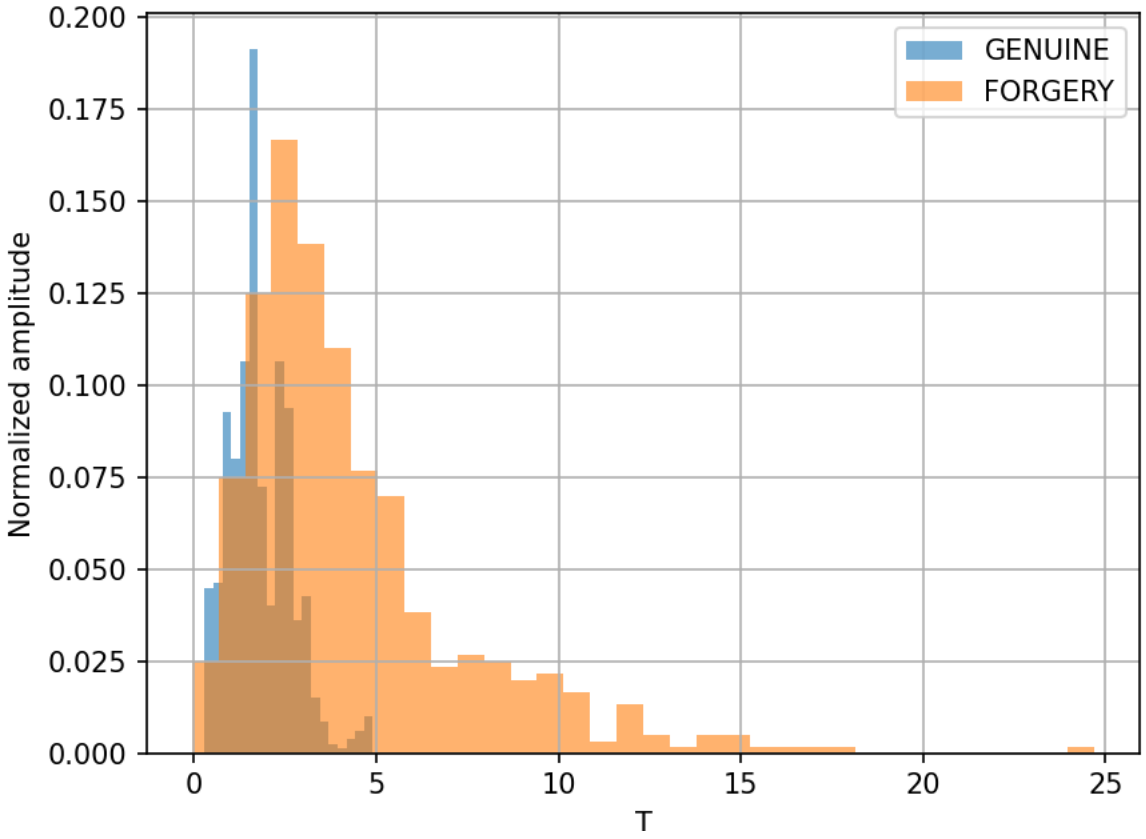
Please, save these two matrixes into the files: *BiosecurIDparameters_genuine.mat* and *BiosecurIDparameters_forgery.mat.* We will use them during the rest of the programming task.

Once you have both matrixes, **for each global feature, you must plot the GENUINE and FORGERY distributions in different figures (histograms)**. The amplitude of the histograms (vertical axe) must be normalized between 0 and 1 (i.e., you must divide by the total number of points of the distribution).

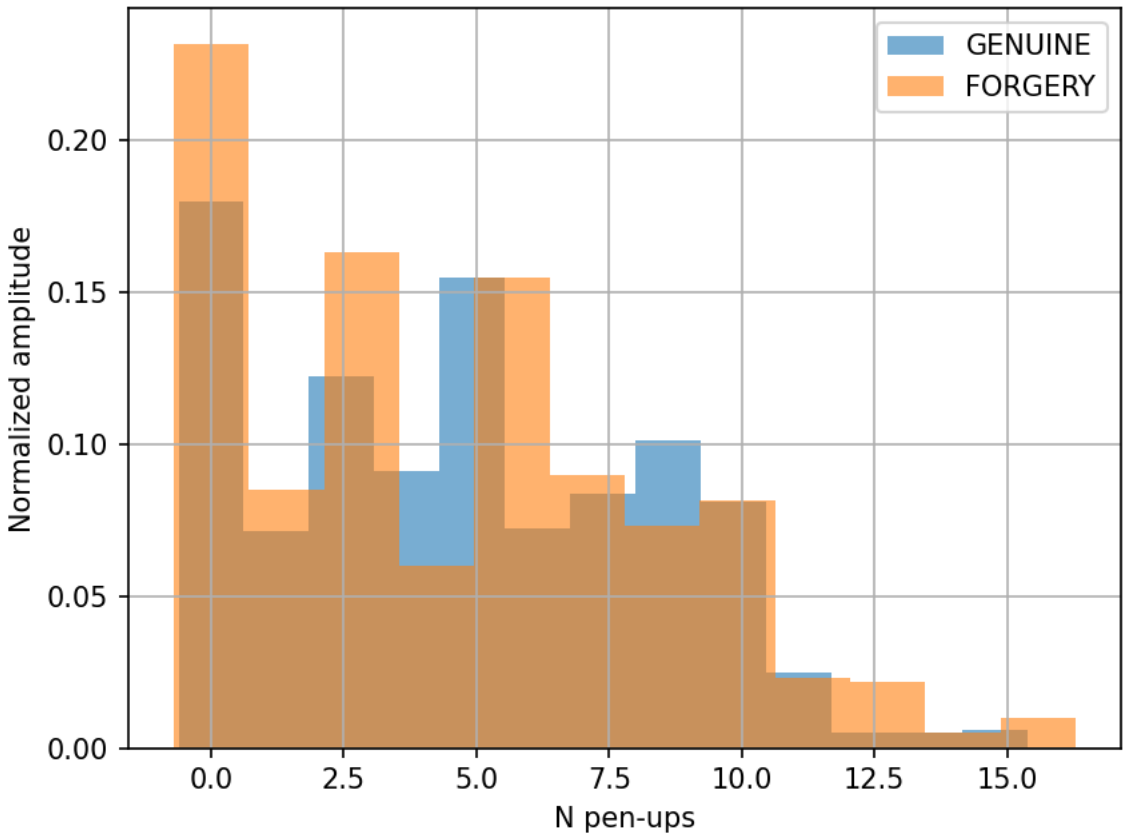
You might find useful the following Python functions: 1) *histogram (*with the value *bins='auto')* included in *numpy* library, and 2) *bar* included in the *matplotlib.pyplot library.*

QUESTION: Plot the GENUINE and FORGERY distributions for each of the 4 global features.

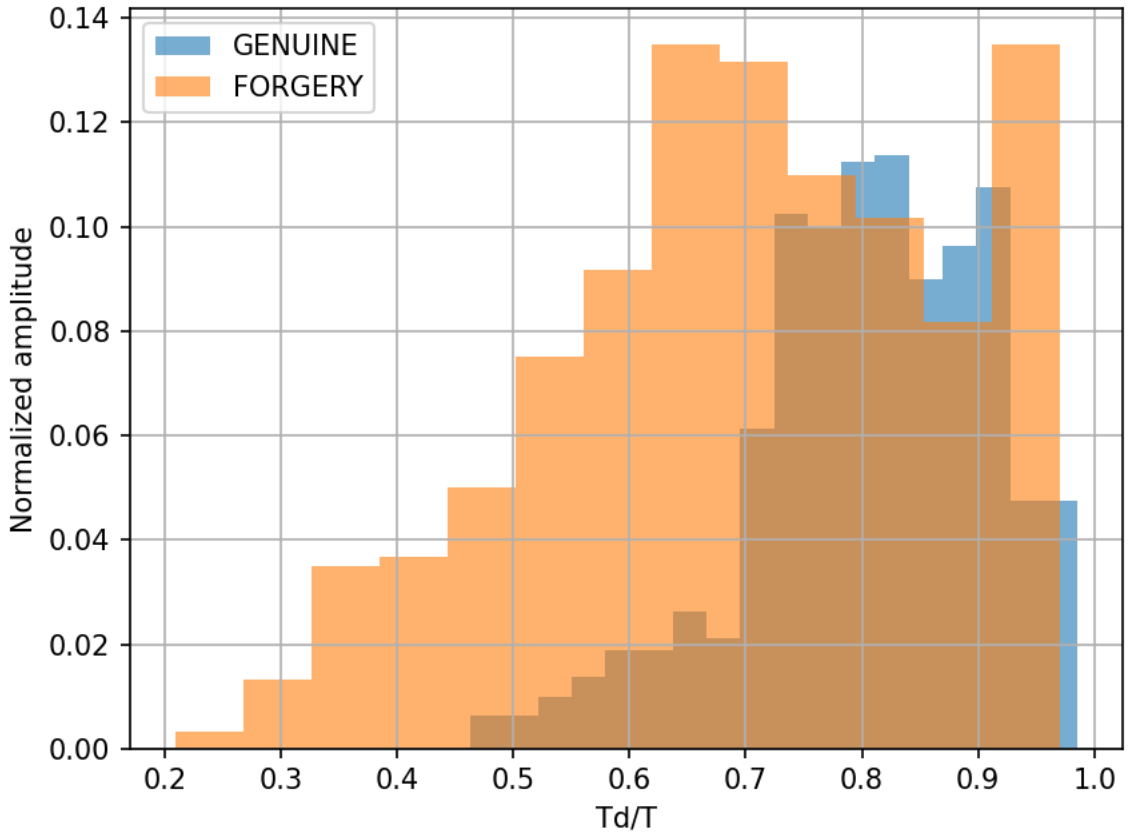
Total duration (in seconds)


GENUINE
FORGERY
Normalized amplitude
T
0.200
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
0
5
10
15
20
25

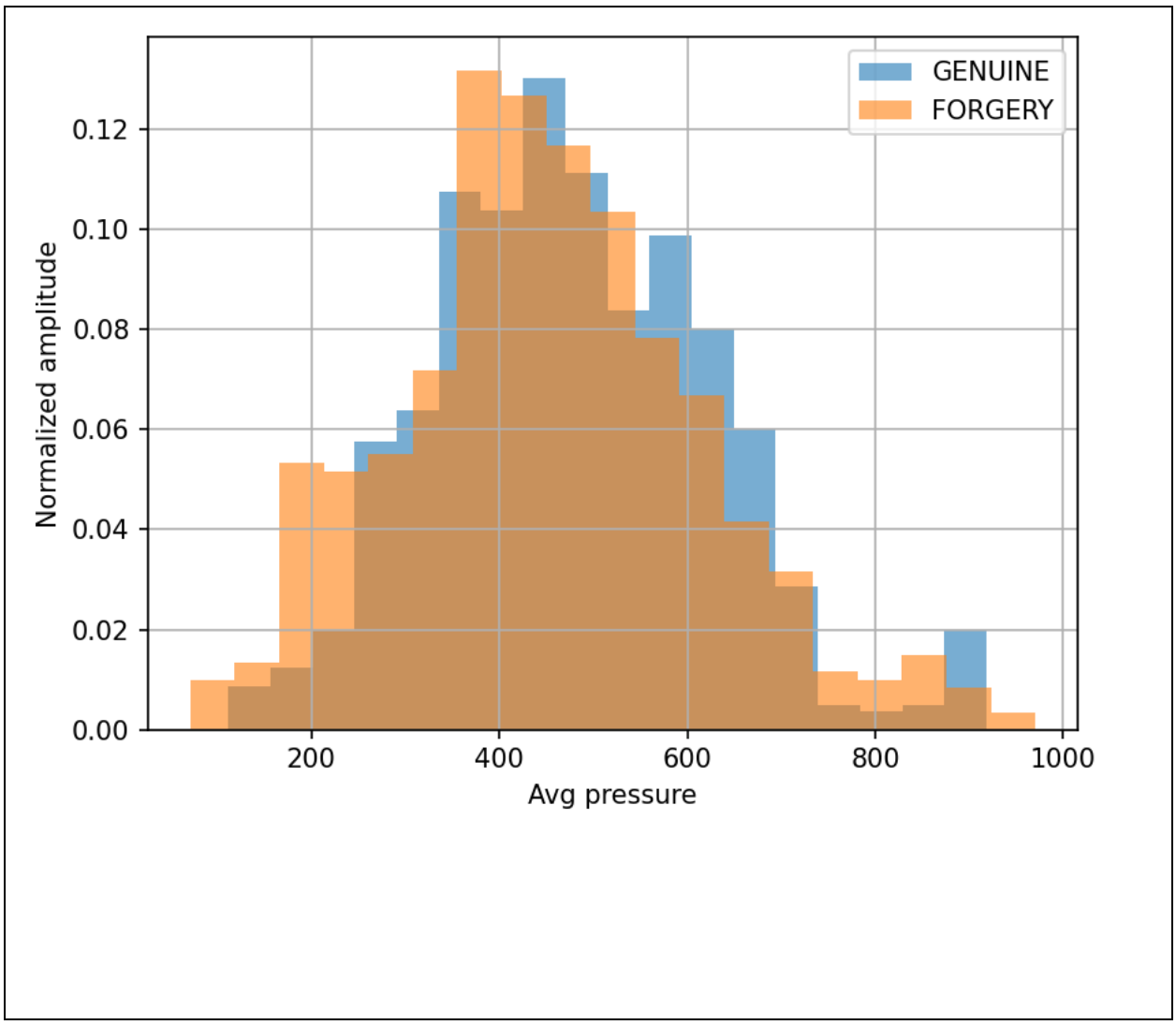
N pen-ups


GENUINE
FORGERY
Normalized amplitude
0.00
0.05
0.10
0.15
0.20
0.0
2.5
5.0
7.5
10.0
12.5
15.0
N pen-ups

T pen-down / T



Average P in pen-down



QUESTION. Analyzing the general GENUINE and FORGERY distributions, are you happy with the proposal of the selected global features? Do you think our on-line handwritten signature verification system will be able to achieve good performance?

From the analysis of the global feature distributions, it can be observed that the selected features provide limited discriminative power between genuine and forgery signatures.

For the total duration, although forgeries tend to present a wider dispersion and sometimes longer durations than genuine signatures, there is still a significant overlap between both distributions. This overlap makes it difficult to reliably distinguish between genuine and forgery samples using this feature alone.

A similar situation occurs with the number of pen-ups, where the distributions of genuine and forgery signatures almost completely overlap. This indicates that this feature is not sufficiently discriminative.

Regarding the ratio of pen-down time (Td/T), there is slightly less overlap compared to the previous features, and some separation between genuine and forgery distributions can be observed. However, the overlap is still considerable.

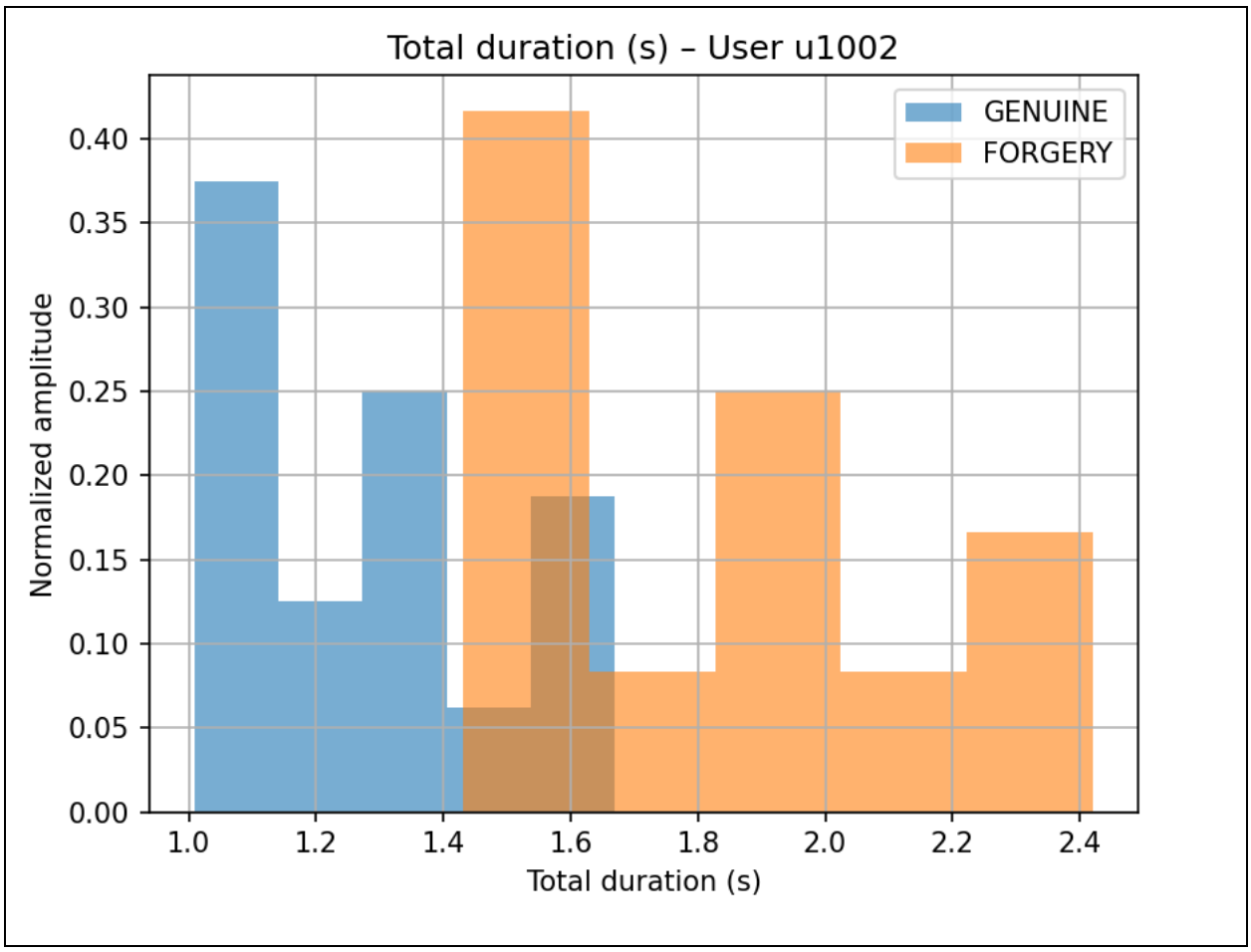
Finally, the average pressure during pen-down also shows a strong overlap between both classes, making it difficult to use as a reliable discriminator.

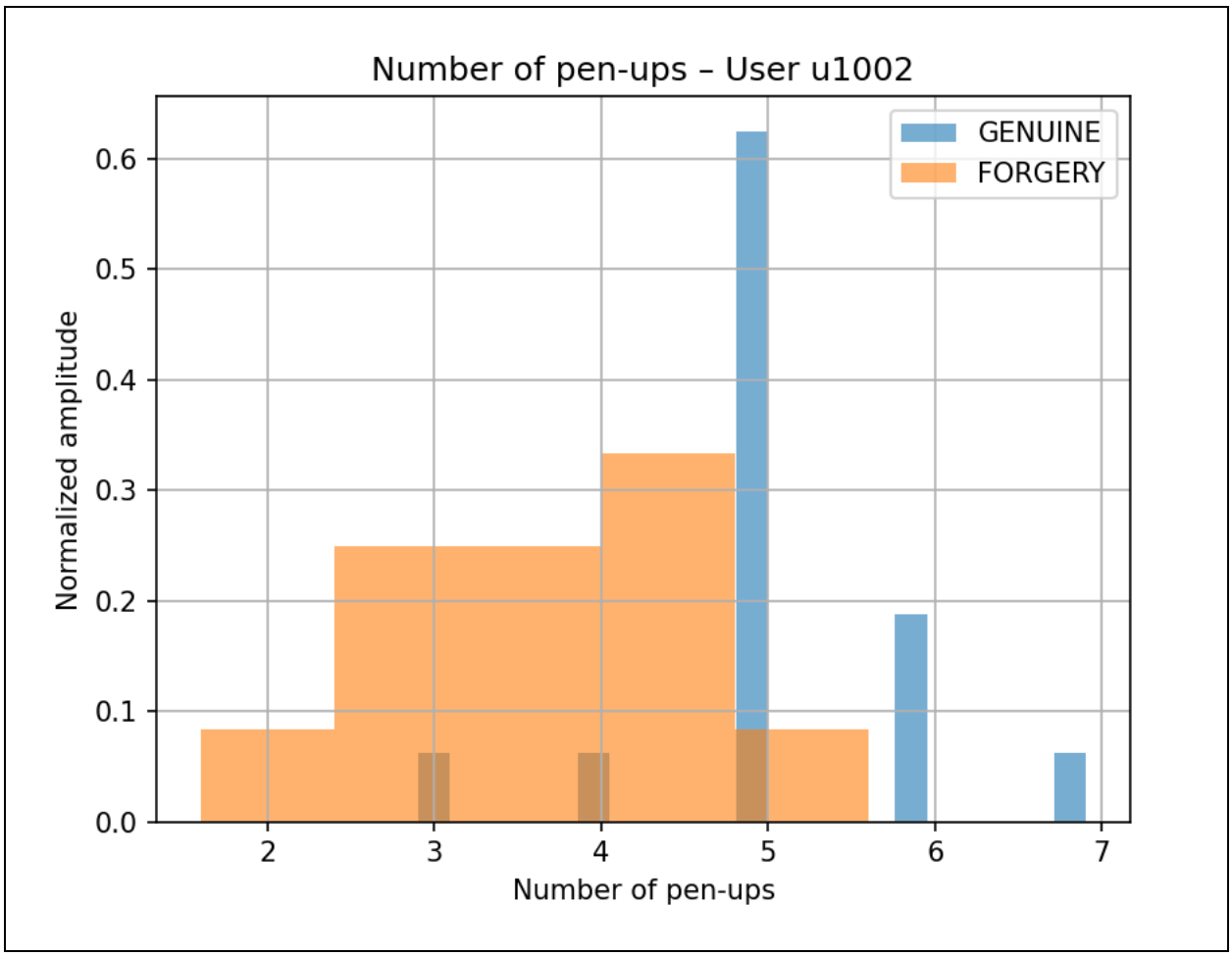
Overall, due to the high degree of overlap in most features, the system based solely on these global features is expected to achieve limited performance.

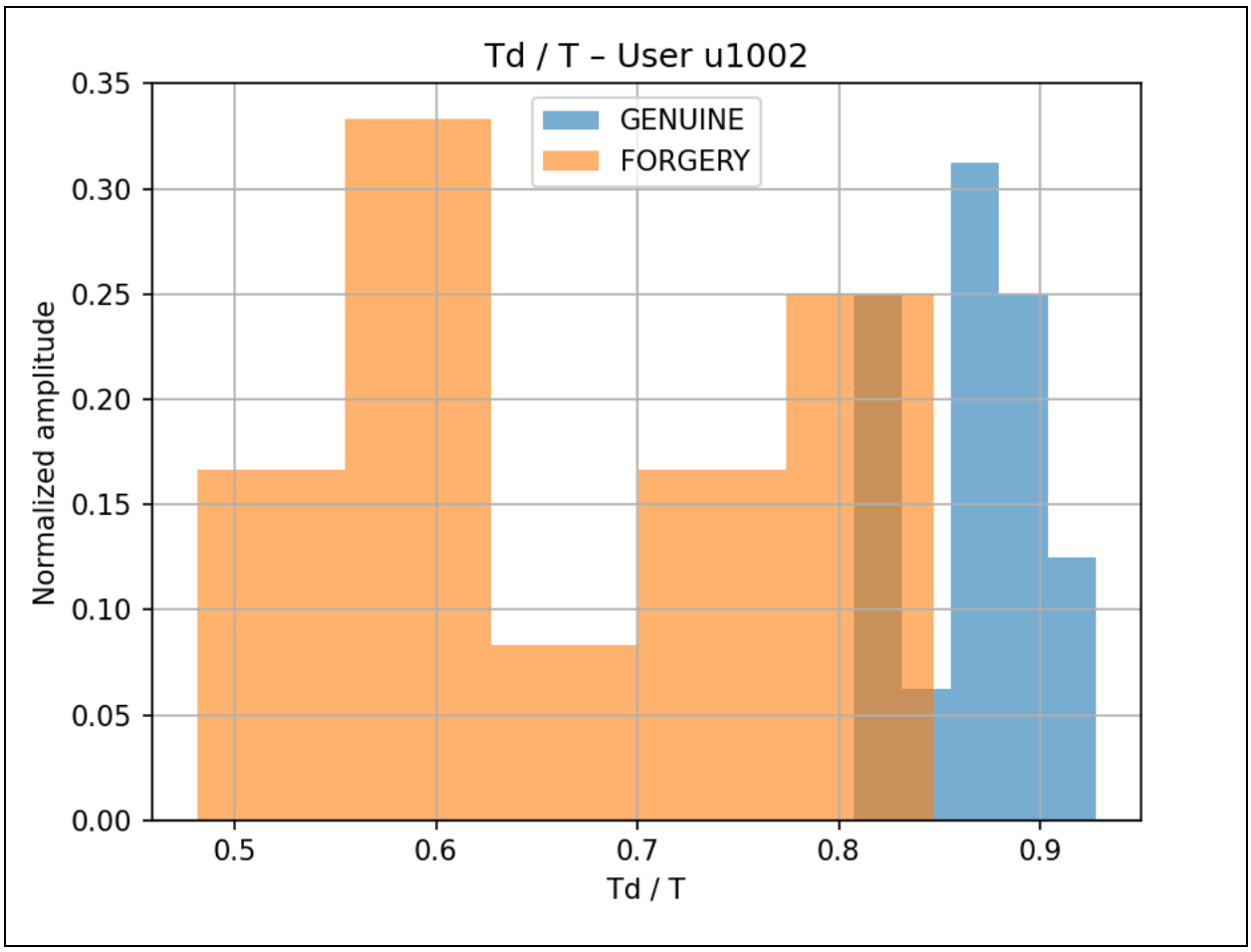
However, when analyzing the distributions for a specific user (e.g., user u1002), the situation improves. In this case, the overlap between genuine and forgery signatures is significantly reduced, and the distributions are more separable. This suggests that, at a user-dependent level, the features may be more discriminative.

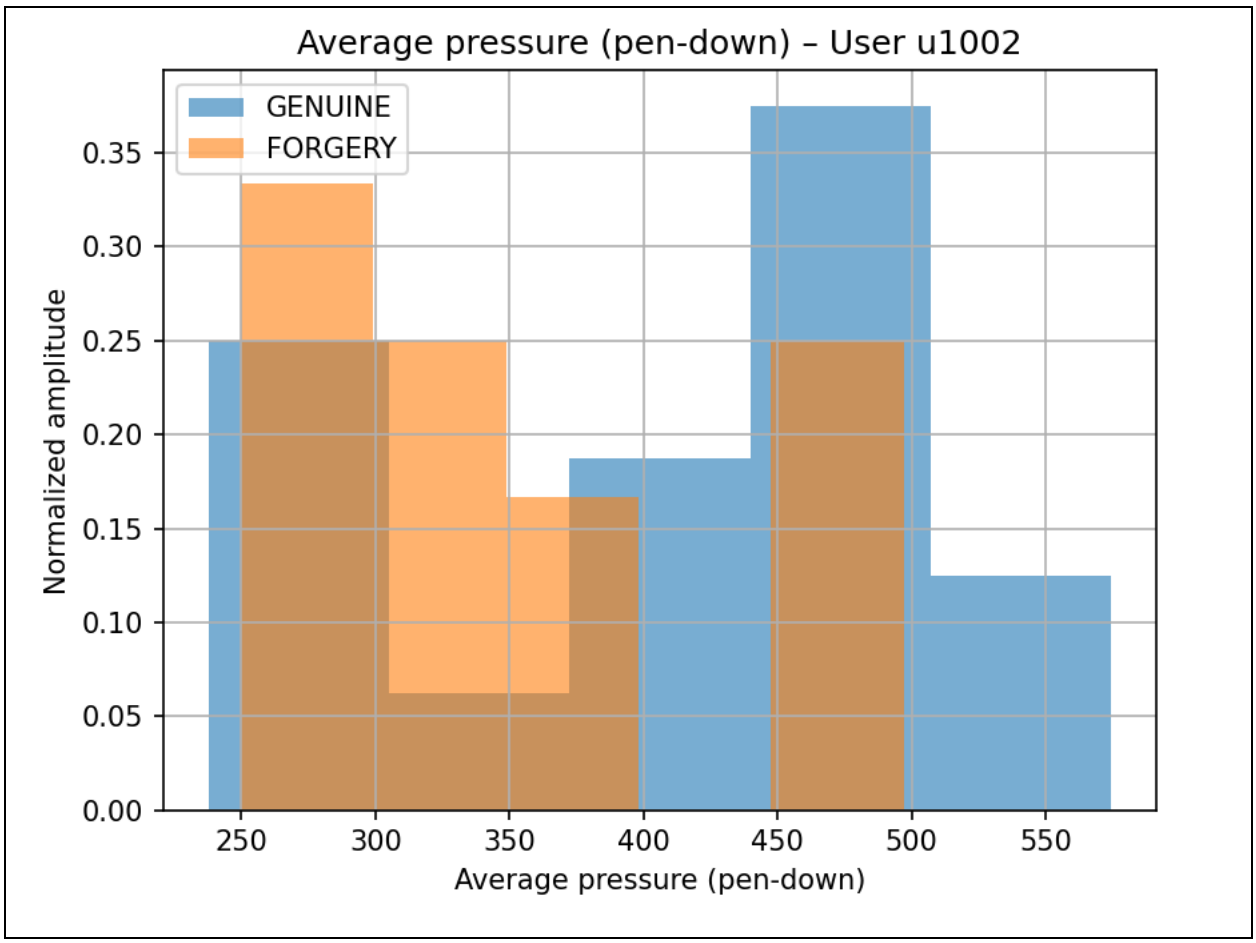
Nevertheless, considering the global statistics across all users, the system is expected to have difficulty achieving good performance, as the general overlap between genuine and forgery distributions remains high.

Finally, please plot the same distributions but consider only the GENUINE and FORGERY signatures of one particular user, e.g., *u1002*. Provide your opinion about these distributions.


Total duration (s) – User u1002
GENUINE
FORGERY
Normalized amplitude
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
1.0
1.2
1.4
1.6
1.8
2.0
2.2
2.4
Total duration (s)


Number of pen-ups – User u1002
GENUINE
FORGERY
Normalized amplitude
0.0
0.1
0.2
0.3
0.4
0.5
0.6
2
3
4
5
6
7
Number of pen-ups


Td / T – User u1002
GENUINE
FORGERY
Normalized amplitude
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0.5
0.6
0.7
0.8
0.9
Td / T



4. Performance Evaluation

We will evaluate the performance of our on-line handwritten signature verification system **considering only the easiest attacks to detect, i.e., random forgery scenario,** where genuine signatures from different users are considered as impostors. **Therefore, for this part we will consider only the *BiosecurIDparameters_genuine.mat* file**. In addition, we will analyze the influence of the number of genuine enrollment signatures available per user (*N=1, N=4 and N=12*) on the system performance. **You might find useful the Python code *Part_4_performance_evaluation_random.py* to complete this section.**

The similarity score between a query/test signature and the enrollment signatures (signatures in the database) **will be the Euclidean distance** between feature vectors (vectors with 4 global features). **The final score will be the average score of the *N* comparisons** (comparison between the query/test sample and the *N* enrollment samples).

First, you must develop the function *Score=Matcher(test,Model)* where:

- *Score:* is the final score of the comparison.
- *test:* is the feature vector of the query/test signature (size equals to 1x4)
- *Model:* is a matrix containing the *N* feature vectors of the signatures enrolled in the database for the claimed user. Therefore, this matrix contains *N*x4 values in which *N* is the number of signatures enrolled for the claimed user (*N=1, N=4 and N=12*).

The *Score* provided by the *Matcher* function must be finally normalized in order to provide higher values for genuine comparisons than forgery comparisons, i.e., *Final_Score = 1 / Score*

The following two types of comparisons must be considered in the analysis:

Genuine Scores: scores obtained when you compare a genuine signature with his/her real enrolled identity (claimed identity = enrolled identity). So, users should be accepted by the system. For each user, you will use N signatures as enrolled samples and the rest for testing, as indicated below:

- For *N=1* we will have SG=15 genuine scores.
- For *N=4* we will have SG=12 genuine scores.
- For *N=12* we will have SG=4 genuine scores.

For each of the scenarios (*N=1, 4, 12*) you must save all the genuine scores into a matrix (with dimension 50xSG). Each of the three matrixes will be stored into a *.mat* file with name: *GenuineScores_N.mat.*

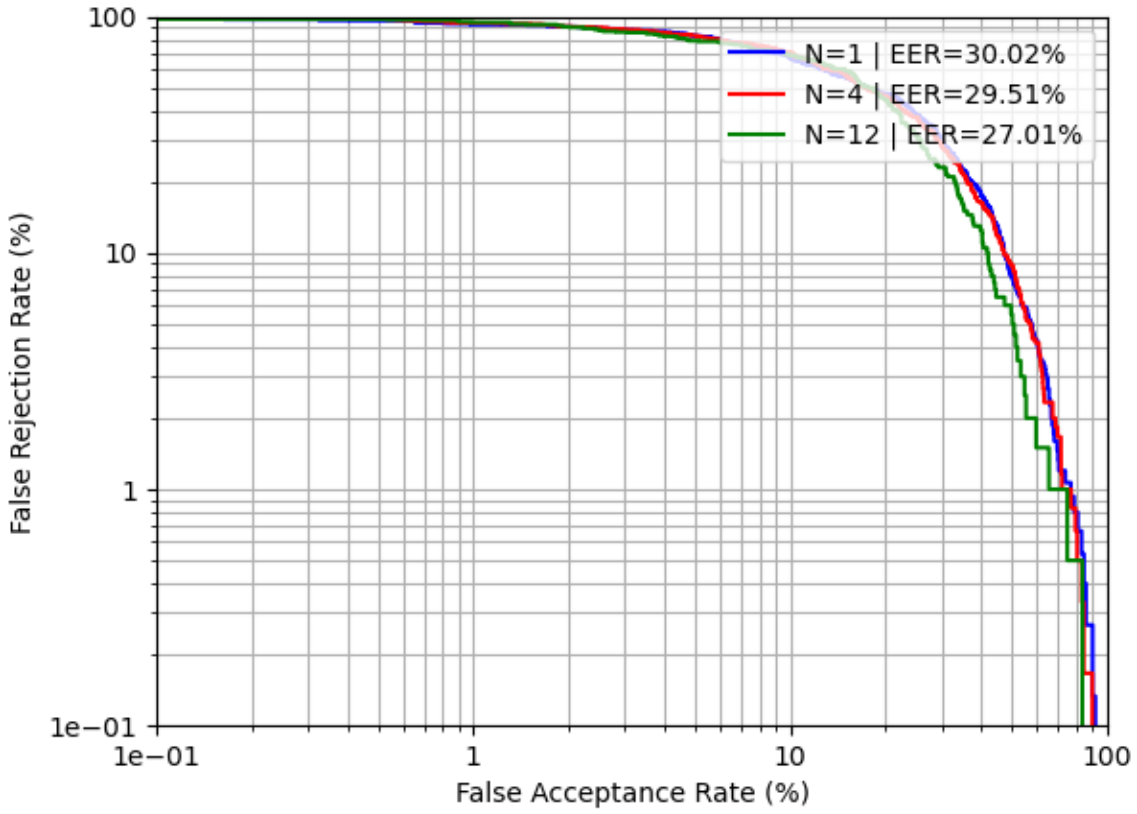
Impostor Scores: scores obtained when you compare a signature from one user with the enrolled samples of other users (claimed identity ≠ enrolled identity). So, these users should be rejected by the system. In this case, the *N* enrolled signatures of the claimed user must be compared with one signature of the remaining users (the first signature, uXXXXs0001_sg0001). Therefore, we will obtain SI=49 impostor scores for each user and each scenario (*N=1, 4, 12*).

For each scenario (*N=1, 4, 12*) these impostor scores will be saved into a matrix with dimensions 50xSI (50x49). Each of the three matrixes will be stored into a *.mat* file with name: *ImpostorScores_N.mat.*

Once we obtain the genuine and impostor scores, we will evaluate the performance of our on-line handwritten signature verification system for each of the three scenarios (*N=1, 4, 12*) as a function of: *FAR/FRR*, *EER* and *DET curves*.

To facilitate the calculation of these metrics, as well as the representation of the DET curve, **you might find useful the Python file *PT1_Metrics_DETcurve_Example.py*, included in Moodle**. The corresponding file provides an example of these metrics using random scores.

QUESTION. Plot the *DET curve* using the genuine and impostor scores stored in their respectively matrixes (for each of the scenarios *N=1, 4, 12*). Indicate the *EER* value.



QUESTION. According to the results, are they reasonable? When do you obtain the best performance?

The obtained results are consistent with the analysis of the feature distributions. Since the global features show a high degree of overlap between genuine and forgery signatures, it is expected that the system does not achieve a high level of accuracy.

The Equal Error Rate (EER) values are relatively high (around 27–30%), which indicates that the system has limited discriminative capability when using only these global features.

Regarding the effect of the number of enrollment signatures, a slight improvement can be observed as N increases:

- N = 1 → 30.02%
- N = 4 → 29.51%
- N = 12 → 27.01%

This behavior is reasonable, since increasing the number of enrollment samples allows the system to better model the intra-user variability, leading to a more robust representation of each user.

However, the improvement is relatively small. This suggests that simply increasing the number of enrollment signatures is not sufficient to significantly enhance performance when the feature set itself is not highly discriminative.

The best performance is obtained for **N = 12**, as expected, although the overall improvement remains limited. This indicates that more informative features or more advanced matching techniques are required to achieve better results.

With all the previous exercises done correctly you can obtain a mark up to 7 points out of 10.

Extra work 1: If you want to obtain a mark up to **8,5 points out of 10** you should complete **all** the following points:

- Think and give a reasonable explanation of some additional global features you can extract from the signatures (at least 4 more). After that, program them and repeat point *4. Performance Evaluation* in order to improve the system performance.
- Obtain a list of the most discriminative features in terms of EER and based on that make combinations of features in order to obtain a better performance.
- Similar to point *4. Performance Evaluation*, run the evaluation using this time the skilled forgeries (i.e., *BiosecurIDparameters_forgery.mat* file) and compared the results with the random forgeries.

In order to improve the performance of the system, several additional global features were proposed, aiming to capture more dynamic information from the signature:

- Maximum velocity: captures the highest speed reached during the signing process.
- Path length: represents the total trajectory of the pen, reflecting the complexity of the signature.
- Mean acceleration: provides information about the dynamics and smoothness of the movement.
- Pressure variance: measures the variability of the applied pressure, which can be characteristic of each user.

After extracting these features and evaluating them individually, their discriminative capability was analyzed using the Equal Error Rate (EER).

The results show that the most discriminative features are:

1. Total duration (T)
2. Number of pen-ups
3. Path length
4. Td/T

While features such as maximum velocity or pressure variance showed poorer performance.

By combining the best features, a significant improvement in performance was achieved. The EER decreased progressively as more informative features were included, reaching the best result when using the top 7 features (EER = 7.51%).

However, adding all features did not lead to further improvement, indicating that some features introduce noise or redundancy.

Finally, when comparing random forgeries and skilled forgeries, the system performs significantly better in the random scenario. Skilled forgeries are more challenging, as they attempt to mimic the genuine signature, leading to higher error rates.

Extra work 2: If you want to obtain a mark up to **10 points out of 10** you should do the following (**directly, without doing the Extra work 1**):

- Develop an on-line signature verification system based on local features (time functions) and Dynamic Time Warping for the Matcher (or similar approaches such as Hidden Markov Models or Recurrent Neural Networks). Repeat the same experimental protocol followed in the programming task but using this new approach.
- You should use the following local features (time functions): x, y, pressure, (and their corresponding first and second derivative).
- You can use the DTW matcher (or others) available in GitHub repositories, for example. Regarding the DTW approach, in most cases in the literature the equation to obtain the score of the time function comparison is $score=e^{-D/k}$, where D is the minimum accumulative distance obtained after using DTW, and K is the number of aligned time samples. You might find this information useful for your proposal.

The inclusion of first- and second-order derivatives allows the system to capture not only the spatial trajectory of the signature, but also its dynamic writing behavior, such as velocity and acceleration patterns. These dynamic characteristics reflect muscular control and are generally more difficult to imitate than the static shape of a signature. As a result, they provide improved discriminative power in

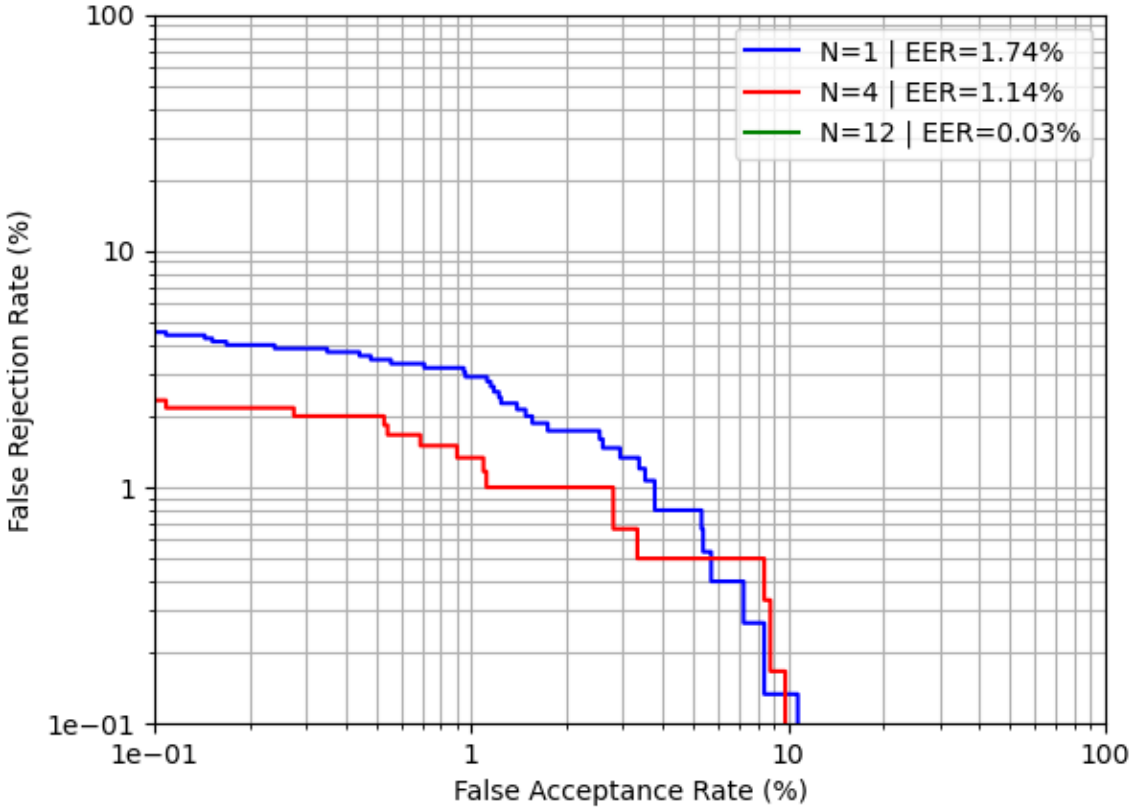
distinguishing genuine signatures from skilled forgeries. The complete feature vector at each time step is therefore a 9-dimensional local descriptor: x, y, p, x', y', p', x'', y'', p''

To perform the signature comparison, Dynamic Time Warping (DTW) is employed. It is helpful since it handles temporal variations in writing speeda, aligns sequences of different length and compensates non linear tome distortions. Given two multivariate time sequences, DTW computes the minimum accumulated distance D between them by finding the optimal alignment path under continuity and monotonicity constraints. The similarity score is calculated via: score = e^{-D / K} with D, the minimum accumulated distance and K, the number of aligned time samples (length of the warping path). Before applying DTW, samples were normalized to avoid value explosions leading to distorted results.

The experiments were performed as in the previous task: The first N genuine signatures are used to build the enrollment model. The remaining genuine signatures are compared against the model to compute genuine scores. Signatures from all other users are compared against the model to compute impostor scores. The final decision threshold is determined by analyzing the distribution of genuine and impostor scores, and performance is evaluated using metrics such as the Equal Error Rate (EER) and DET curves.

In order to accelerate DTW which runs slowly for long samples, we distributed the user comparisons to an number of selected kernels that run in parallel. This reduces the computational time drastically.

The DET curves look as follows:



The usage of local features greatly reduces the EER. Especially for N=12, results are almost perfect since there are many available samples enrolled for one user. It makes the model more robust, resulting in lower errors. The experiment shows that the selected features are highly discriminative. The curves drop sharply around FAR ≈ 5–10%. This means that genuine and impostor distributions are well separated. Only a small threshold shift dramatically reduces errors. We consider this is as a good biometric system.